

TalentLock Regression Tests

What the automated regression suite does, how it is organised, and how to run it.

Generated: July 2026 | Branch: cursor/regression-tests-9a23

In short: Regression tests automatically re-check that TalentLock still works correctly after every code change. They cover API security, marketplace flows, billing limits, AI features, messaging, dashboards, frontend utilities, and legacy validation scripts — catching bugs before they reach production.

1. What is a regression test?

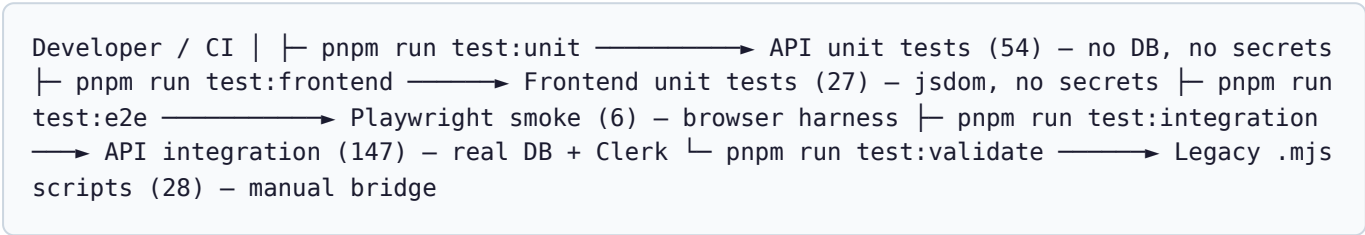
A **regression test** verifies that existing behaviour has not broken after a change. In TalentLock, the suite runs hundreds of automated checks against the API server, frontend components, and (optionally) a live browser.

Each test follows a simple pattern:

- 1. **Arrange** — set up inputs, auth tokens, or database fixtures.
- 2. **Act** — call an API endpoint, render a component, or load a page.
- 3. **Assert** — confirm the response status, JSON shape, UI text, or security header matches expectations.

If a developer changes code that breaks a previously working feature, the matching test fails immediately — in local development or in GitHub Actions CI — before the change is merged.

2. Test architecture overview



3. Test layers and what they verify

3.1 API unit tests — 54 tests (CI-safe)

Location: artifacts/api-server/tests/unit/

Runner: Vitest | **Command:** pnpm run test:unit

Fast, isolated tests for pure functions and business logic. No database or network required.

Test file	What it checks
sanitise.test.ts	HTML stripping from user text; safe rich-text tags allowed
completenessUtils.test.ts	Profile completeness score 0–100; ≥60% gate for Talent Vault
paginationUtils.test.ts	Page bounds and totalPages calculation
plans.test.ts	All 5 subscription plan IDs exist with correct limits
subscriptionGating.test.ts	Token usage aggregation and monthly reset date logic
searchUtils.test.ts	Search query sanitisation (SQL injection safe)
accessControl.test.ts	Access-control helper exports present
cruiseModeUtils.test.ts	Cruise Mode pre-filter, blackout windows, prompt validation
talentSearchUtils.test.ts	Talent Search pre-filter and prompt validation
professionContext.test.ts	Teaching profession context builder
securitySource.test.ts	Security hardening source patterns (Phase 2 checks)
legacyCoverage.test.ts	Every legacy script and Vitest port file exists on disk

3.2 API integration tests — 147 tests (needs secrets)

Location: `artifacts/api-server/tests/integration/`

Runner: Vitest + Supertest | **Command:** `pnpm run test:integration`

Hits the real Express API against a test database and Clerk test users. Tests are automatically **skipped** when `DATABASE_URL`, `CLERK_SECRET_KEY`, or `CLERK_PUBLISHABLE_KEY` are not configured.

Area	What it checks
Phase 1 Auth & Security	IDOR guards (401/403/200 per route), security headers, admin CSRF login, input sanitisation, GDPR deletion
Phase 2 Core marketplace	Users, freelancers/vault, employers, jobs, bookings, meetings, agreements, reviews, availability
Phase 3 Billing & tokens	Subscription plans, quota enforcement (402 PLAN_LIMIT / TOKEN_LIMIT), per-conversation token usage
Phase 4 AI & automation	AI endpoint auth guards, rate suggestions, proposals, match explanations, Cruise Mode & Talent Search config
Phase 5 Messaging & docs	Direct messaging dedup/rate limits, notification inbox/read ordering, document verification
Phase 6 Dashboard & teams	Role-specific dashboard panels, earnings intelligence, team access (Enterprise plan), teaching profiles
Phase 8 Legacy ports	Agreement health/redline/summary/download, product gaps, calendar, reviews submit, email notifications

3.3 Frontend unit tests — 27 tests (CI-safe)

Location: `artifacts/talentlock/tests/`

Runner: Vitest + Testing Library | **Command:** `pnpm run test:frontend`

Test file	What it checks
<code>rateFormatUtils.test.ts</code>	Rate type labels (hourly, daily, fixed, etc.)
<code>contractHealthUtils.test.ts</code>	Agreement health grade boundaries (A-F)
<code>agreementSummaryUtils.test.ts</code>	Summary section ordering
<code>planErrorUtils.test.ts</code>	402 TOKEN_LIMIT shows inline error; PLAN_LIMIT redirects to /pricing
<code>exclusivityCopy.test.ts</code>	Pending bookings never display "locked" copy
<code>VerifiedEmployerBadge.test.tsx</code>	Badge hidden when employer is unverified
<code>PaginationControls.test.tsx</code>	Prev/next disabled at first/last page
<code>CompletenessBanner.test.tsx</code>	Missing profile fields shown to user

3.4 Playwright E2E smoke — 6 tests

Location: `e2e/smoke.spec.ts` | **Command:** `pnpm run test:e2e`

Browser-level smoke tests. One harness test always runs in CI. The remaining 5 tests (landing page, health API, sign-in, pricing, admin login) run only when `E2E_FORCE=1` or `E2E_BASE_URL` is set with dev servers running.

3.5 Legacy validation bridge — 28 scripts

Location: `scripts/test-regression.mjs` | **Command:** `pnpm run test:validate`

Before the Vitest suite existed, TalentLock used standalone `validate-*.mjs` scripts run manually. The bridge orchestrator still runs all 28 legacy scripts for backwards compatibility. Each script is mapped to a modern Vitest suite in `artifacts/api-server/tests/regression/legacy-map.ts`.

4. What regression tests protect against

- **Security regressions** — users accessing other users' bookings, agreements, or messages (IDOR)
- **Auth bypass** — admin routes without CSRF, missing security headers
- **Billing bugs** — quota limits not enforced; wrong 402 error shape
- **Data integrity** — profile completeness score wrong; vault listing incomplete profiles
- **API contract drift** — response shapes changing without notice
- **Frontend UX bugs** — wrong error handling on plan limits; broken pagination
- **Feature regressions** — agreements, AI, messaging, notifications breaking silently

5. Commands reference

```
pnpm run typecheck      # Full workspace TypeScript check
pnpm run test            # API unit + frontend unit (CI-safe)
pnpm run test:unit       # API server unit tests only
pnpm run test:frontend   # Frontend unit tests only
pnpm run test:integration # API integration (needs .env secrets)
pnpm run test:e2e        # Playwright smoke harness
pnpm run test:validate   # Legacy .mjs scripts (needs .env secrets)
pnpm run test:all        # unit + frontend + full API vitest suite
```

6. Local setup (secrets never committed)

1. Copy `.env.example` → `.env` locally.
2. Fill in: `DATABASE_URL`, `CLERK_SECRET_KEY`, `CLERK_PUBLISHABLE_KEY`.
3. Optional: `ADMIN_PASSWORD`, `CSRF_SECRET`, demo Clerk user IDs.
4. Run `pnpm run test:integration` to execute all 147 integration tests.

`.env` and `.env.test` are gitignored. Never commit secrets to the repository.

7. CI pipeline (GitHub Actions)

Workflow file: `.github/workflows/test.yml`

Job	When	Steps
CI	Every PR and push to main	typecheck → API unit (54) → frontend unit (27) → Playwright E2E harness
Integration	Push to main + nightly 03:00 UTC	Verify secrets → DB schema push → API integration (147)

Required GitHub secrets for integration job:

- `DATABASE_URL_TEST` — Neon test database connection string
- `CLERK_SECRET_KEY` / `CLERK_PUBLISHABLE_KEY`
- `CSRF_SECRET` / `ADMIN_PASSWORD`

Vitest JUnit XML reports are uploaded as GitHub Actions artifacts after each run.

8. Test count summary

Layer	Tests	Runs in CI without secrets?
API unit	54	Yes — every PR
Frontend unit	27	Yes — every PR
Playwright E2E	1 pass + 5 skip	Yes — harness only
API integration	147	Main + nightly (with secrets)
Legacy scripts	28 scripts	Manual / local only
Total Vitest	203	81 pass / 147 skip without .env

9. Legacy script → Vitest migration map (Phase 8)

28 legacy scripts are tracked in `legacy-map.ts`. Priority P0 scripts cover auth, security, and completeness utils. P1 covers agreements, AI, availability, and tokens. P2 covers teams, notifications, reviews, and earnings.

Priority	Legacy script examples	Modern Vitest suite
P0	validate-auth-hardening.mjs	integration/auth/idor.test.ts
P0	validate-security-hardening.mjs	integration/security/middleware.test.ts
P1	validate-contract-health-score.mjs	integration/agreements/health.test.ts
P1	validate-ai-proposal-generator.mjs	integration/ai/proposal.test.ts
P2	verify-earnings-intelligence.mjs	integration/dashboard/earnings.test.ts
P2	verify-reviews-all.mjs	integration/reviews/submit.test.ts

10. How a typical integration test works

1. Test file calls `integrationEnvReady()` — skips entire suite if secrets missing.
2. `apiClient.ts` mints a real Clerk JWT for demo employer or freelancer.
3. Supertest sends HTTP request to the in-process Express app.
4. Response status, JSON body, and headers are asserted.
5. For quota tests: asserts HTTP 402 with `{ error, code, planNeeded }` shape.
6. For IDOR tests: same route called as wrong user → expects 403.